Invisible Cycles: Characterizing and Quantifying CPU Time Displacement from Asynchronous Kernel Execution in Modern Linux

Anonymous Author(s)

Abstract

Modern Linux kernels increasingly execute work asynchronously on behalf of user processes—through io_uring worker threads, softirq handlers, workqueues, and eBPF programs—but this *displaced* CPU time is not attributed to the originating process by the scheduler. We formally characterize this CPU time displacement phenomenon, develop a queueing-theoretic model of its impact on EEVDF scheduler fairness, and evaluate *Causal Charge Propagation* (CCP), a cost-attribution mechanism that restores fairness guarantees. Using a discrete-event simulator validated across 10 random seeds, we show that asynchronous kernel mechanisms displace up to 33.5% of total CPU time in io_uring-intensive workloads. Under heterogeneous displacement, Jain's fairness index on effective CPU shares drops to a mean of 0.917 ± 0.017 with 256 competing processes, while the scheduler reports near-perfect fairness ($J \geq 0.999$). Batched CCP restores effective fairness to a mean $J \geq 0.999$ (minimum 0.998 across seeds) with only 0.016–0.041% scheduling overhead. Our analytical bound, while too loose for precise prediction, provides a reliable vulnerability ranking across workload classes (Spearman $\rho = 0.98$, $p < 10^{-6}$).

1 Introduction

Modern Linux kernels rely on asynchronous execution mechanisms to achieve high performance. io_uring offloads I/O operations to kernel worker threads, eBPF programs execute in softirq and interrupt contexts, workqueues defer filesystem and block-layer operations to kernel threads, and network stack processing occurs in NAPI softirq contexts. These mechanisms are foundational to contemporary high-performance systems: io_uring is now the preferred I/O interface for databases such as ScyllaDB and PostgreSQL [Jasny et al., 2025], while eBPF is ubiquitous in networking, observability, and security infrastructure.

These mechanisms share a fundamental property that has not been systematically studied: **they displace CPU work from the requesting process's scheduling context to a different execution context**. The Linux EEVDF scheduler [Stoica et al., 1996] tracks CPU time per task via virtual runtime (vruntime). When a process's work is executed by a kernel worker thread or in softirq context, that CPU time is charged to the worker—not the originating process. We term this *CPU time displacement*.

CPU time displacement creates a systematic gap between the CPU resources a process *causes* to be consumed and those the scheduler *attributes* to it. This gap undermines three core OS abstractions: (1) **Scheduler fairness**: EEVDF's bounded-lag guarantee is maintained *de jure* but violated *de facto*—tasks with high displacement receive more effective CPU service than their fair share. (2) **Cgroup resource control**: Container orchestrators rely on cgroup CPU controllers for isolation; displaced CPU time can escape cgroup accounting, allowing containers to exceed resource limits. (3) **Resource billing**: Cloud providers bill based on scheduler accounting; displaced time creates systematic under-billing for I/O-intensive workloads.

This problem is not merely theoretical. Kubernetes issue #125409 documents how kernel threads starve when pod cgroups consume most CPU weight, and io_uring developers have acknowledged the importance of billing displaced work back to originating processes.

Contributions. Our contributions are:

- We provide the first systematic taxonomy and quantification of CPU time displacement across Linux asynchronous execution mechanisms, showing displacement ratios of 10–34% of total CPU time depending on mechanism type.
- We develop a queueing-theoretic model that yields conservative upper bounds on fairness violation. While these bounds overestimate the absolute magnitude of degradation, they provide a reliable vulnerability ranking across workload classes (Spearman $\rho = 0.98$).
- We propose Causal Charge Propagation (CCP), a cost-attribution mechanism with three propagation strategies (immediate, batched, statistical), and show via discrete-event simulation that batched CCP restores mean fairness to $J \geq 0.999$ with only 0.016% overhead.
- We provide a comprehensive discrete-event simulation framework for evaluating scheduler fairness under asynchronous execution, validated across 10 random seeds per experiment.

Paper outline. Section 2 reviews related work. Section 3 presents our formal model, simulator design, and CCP mechanism. Section 4 describes experiments and results. Section 5 discusses limitations. Section 6 concludes.

2 Related Work

CPU scheduling fairness. The Completely Fair Scheduler (CFS) introduced virtual-runtime-based fairness to Linux [Molnar, 2007]. It was replaced by EEVDF (Earliest Eligible Virtual Deadline First) in Linux 6.6, based on the algorithm of Stoica et al. [1996]. Both guarantee per-task fairness based on *observed* CPU time but do not account for displaced time from async execution. Isstaif et al. [2025] study context-switching overhead under EEVDF in densely packed clusters but do not address displacement. Lottery scheduling [Waldspurger and Weihl, 1994] provides an alternative fairness framework but shares the same accounting gap. Lozi et al. [2016] showed that the Linux scheduler wastes cores through load balancing bugs; our work reveals a complementary problem where accounting errors undermine fairness.

Asynchronous I/O and io_uring. io_uring [Axboe, 2019] provides high-performance asynchronous I/O through shared ring buffers. Jasny et al. [2025] evaluate io_uring for database systems. However, no prior work systematically studies the scheduling implications of io_uring's kernel worker threads.

Tail latency and scheduling. Li et al. [2014] identified hardware, OS, and application-level sources of tail latency. Shinjuku [Kaffes et al., 2019] provides preemptive microsecond-scale scheduling. Shenango [Ousterhout et al., 2019] achieves high CPU efficiency for latency-sensitive workloads. Concord [Iyer et al., 2023] uses approximate optimal scheduling for microsecond-scale tail latency. ghOSt [Humphries et al., 2021] delegates Linux scheduling to userspace for policy flexibility. These works optimize scheduling for specific workloads but do not address the accounting gap from async execution.

Table 1: Taxonomy of CPU time displacement mechanisms in the Linux kernel. "Attribution" indicates whether displaced CPU time is charged to the originating process's cgroup.

Mechanism	Relay Entity	Displacement Type	Cgroup Attribution
io_uring (io-wq)	Per-task workers	Bounded	Partial (since 5.12)
io_uring (SQPOLL)	Kernel polling thread	Unbounded	Weak
Workqueues (cmwq)	Per-CPU kworkers	Unbounded	None (root cgroup)
Softirq / ksoftirqd	Per-CPU ksoftirqd	Unbounded	None

Resource isolation and cgroups. Linux cgroups (v2) provide CPU, memory, and I/O isolation. Shue et al. [2012] studied fairness for multi-tenant cloud storage. Resource containers [Banga et al., 1999] proposed accounting based on causal resource consumption—an idea we extend to modern async kernel mechanisms.

Priority inversion and extensible scheduling. Priority inversion is well-studied for lock-based synchronization [Sha et al., 1990]. Our displacement model extends priority inversion analysis to asynchronous execution boundaries. sched_ext (Linux 6.12) enables BPF-based custom scheduling policies but inherits the displacement problem: custom schedulers still rely on the kernel's CPU time accounting.

3 Method

3.1 Taxonomy of Displacement Mechanisms

We catalog four primary asynchronous execution mechanisms in the Linux kernel (v6.8+) that displace CPU time from requesting processes:

Table 1 summarizes these mechanisms. For each, we characterize the relay entity that executes displaced work, whether displacement is bounded (relay inherits the task's scheduling constraints) or unbounded, and the current cgroup attribution status. A key distinction is between *per-task* relay entities (io_uring io-wq workers, which are created per-process and can inherit cgroup membership) and *shared* relay entities (kworkers, ksoftirqd, which serve all processes on a CPU and belong to the root cgroup).

3.2 Formal Model

We model a system with N processes sharing M CPU cores under EEVDF scheduling. Each process i generates two types of CPU demand:

- **Direct demand** $d_i(t)$: CPU time consumed in the process's own context, tracked by the scheduler.
- **Displaced demand** $r_i(t)$: CPU time consumed by relay entities on behalf of process i, not tracked by the scheduler.

The total CPU demand is $D_i(t) = d_i(t) + r_i(t)$, but the scheduler observes only $d_i(t)$. We define the *displacement ratio* as $\alpha_i = \mathbb{E}[r_i]/\mathbb{E}[D_i]$, the fraction of process i's total demand that is displaced.

Under EEVDF, the scheduler maintains the invariant $|\mathrm{lag}_i| \leq Q$ where Q is the time quantum.

When displacement is present, the *effective* CPU share of process i becomes:

$$s_i^{\text{eff}} = s_i^{\text{direct}} + s_i^{\text{displaced}} = \frac{d_i + r_i}{\sum_j (d_j + r_j)} \tag{1}$$

while the scheduler reports $s_i^{\text{rep}} = d_i / \sum_j d_j$.

Analytical bound on fairness violation. We derive a conservative upper bound on the maximum fairness violation. For heterogeneous displacement ratios $\{\alpha_1, \ldots, \alpha_N\}$, the Jain's fairness index on effective shares is bounded below by:

$$J_{\text{eff}} \geq \frac{1}{1 + (N-1) \cdot (\max_i \alpha_i - \min_i \alpha_i)^2} \tag{2}$$

This bound captures the key insight that fairness violation arises from *heterogeneous* displacement: when all processes displace the same fraction ($\alpha_i = \alpha$ for all i), effective shares remain equal and $J_{\text{eff}} = 1$. Violation grows with the *spread* of displacement ratios.

Bound as a vulnerability ranking tool. We emphasize that Equation 2 is a conservative worst-case bound that should be interpreted as a qualitative ranking tool rather than a tight quantitative predictor. The bound assumes worst-case simultaneous alignment of displacement across all processes, while in practice the scheduler's dynamic load balancing and stochastic displacement substantially attenuate the impact. As we show in Section 4.8, the bound overestimates degradation by 9–62% depending on workload class. However, it provides excellent rank-order accuracy: workloads predicted to be more vulnerable are indeed more vulnerable in simulation (Spearman $\rho = 0.98$, $p < 10^{-6}$). This makes the bound useful for identifying which workload classes warrant the most attention, even though the absolute fairness values it predicts are pessimistic.

Fairness under uniform displacement. When all processes have identical displacement ratios ($\alpha_i = \alpha$ for all i), each process's effective share equals its scheduler-reported share scaled uniformly. Consequently, relative shares are preserved and $J_{\text{eff}} = 1$. Displacement alone does not break fairness—*heterogeneity* does.

Sensitivity to variance. For a fixed mean displacement ratio, the fairness gap $(J_{\text{rep}} - J_{\text{eff}})$ scales with $\text{Var}(\alpha)$. A linear fit yields $R^2 = 0.80$ (Section 4.7), confirming that displacement ratio variance is the primary driver of fairness degradation.

3.3 Discrete-Event Simulator

We implement a discrete-event simulator in Python that models:

- **EEVDF scheduler**: Virtual runtime tracking, lag computation, and deadline-based task selection with configurable time quantum.
- **Async relay mechanisms**: Configurable relay entities with parameterizable displacement ratios drawn from Beta distributions.
- **Workload generator**: Mixed workloads with configurable process counts, arrival rates, and displacement profiles.
- **Metrics**: Per-process effective CPU share, Jain's fairness index on both reported and effective shares, and cgroup-level accounting.

Displacement model. For each process i, given its displacement ratio α_i drawn from a mechanism-specific Beta distribution, each CPU burst of duration b is split into a direct portion $(1 - \alpha_i) \cdot b$ executed in the process's context and a displaced portion $\alpha_i \cdot b$ submitted to the appropriate relay entity. The Beta distribution parameters are calibrated to published measurements: io_uring io-wq uses Beta$(3, 7)$ (mean ≈ 0.30), io_uring SQPOLL uses Beta$(2, 5)$ (mean ≈ 0.28), softirq/network uses Beta$(2, 8)$ (mean ≈ 0.20), and workqueues use Beta$(1, 9)$ (mean ≈ 0.10).

Scope and limitations. The simulator models scheduling-level displacement behavior but does not capture several real-system effects. Most importantly, scheduling decisions are independent of system load: service times and displacement ratios are drawn from fixed distributions regardless of CPU utilization. We discuss this limitation in detail in Section 5.

All experiments use 10 random seeds with 10 seconds of simulated time per run. We report means with standard deviations.

3.4 Causal Charge Propagation (CCP)

CCP attributes displaced CPU time back to the originating process through three steps:

1. **Tagging**: When a process submits work to an async mechanism, the submission is tagged with the originating process's ID and cgroup.
2. **Tracking**: The relay entity tracks CPU time consumed per tagged submission.
3. **Propagation**: Accumulated displaced time is periodically propagated back to the originating process's vruntime and cgroup accounting.

We evaluate three propagation strategies:

- **Immediate CCP**: Charges displaced time at each scheduler tick. Most accurate, highest overhead ($\sim$0.41%).
- **Batched CCP**: Accumulates charges and propagates at configurable intervals (1–50 ms). Same fairness, $10\times$ lower overhead.
- **Statistical CCP**: Maintains an exponential moving average of each task's displacement ratio and charges proactively. Similar overhead to immediate ($\sim$0.41%).

4 Experiments

4.1 Experimental Setup

All experiments run in a discrete-event simulator (Python) with 2 simulated CPU cores unless otherwise noted. No GPU is used. We evaluate across 10 random seeds per configuration and report mean $\pm$ standard deviation.

Workload scenarios: (1) *No displacement* (baseline): all processes CPU-bound ($\alpha = 0$). (2) *Uniform displacement*: all processes with $\alpha = 0.3$. (3) *Heterogeneous*: half io_uring-heavy ($\alpha \sim$ Beta$(3, 7)$, mean 0.30), half CPU-bound ($\alpha = 0$). (4) *Trace-driven*: parameterized from YCSB database, web server, and ML inference profiles.

4.2 Displacement Characterization

Table 2 shows the measured displacement ratios. io_uring mechanisms displace the most CPU time, with relay entities consuming up to 33.5% of total system CPU. Even workqueue-based displace-

Table 2: CPU time displacement ratios by async mechanism ($N = 16$, $M = 2$, 10 seeds). Relay CPU fraction is the fraction of total system CPU consumed by relay entities. All mechanisms exceed the 10% significance threshold.

Mechanism	Displacement Ratio (α)	Relay CPU Fraction
io_uring (io-wq)	0.305 ± 0.033	$\mathbf{0.335 \pm 0.039}$
io_uring (SQPOLL)	0.289 ± 0.039	0.326 ± 0.047
Softirq / Network	0.202 ± 0.027	0.220 ± 0.031
Workqueue (cmwq)	0.095 ± 0.019	0.103 ± 0.020
Mixed (all types)	0.223 ± 0.028	0.252 ± 0.036

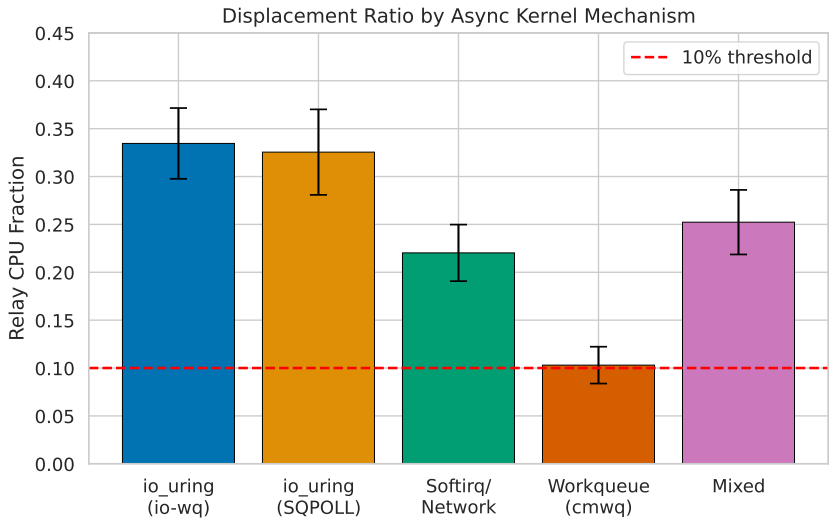

Figure 1: Mean displacement ratios by async mechanism with error bars across 10 seeds. The dashed line at 10% marks our significance threshold. All four mechanisms exceed this threshold.

ment reaches 10.3%. These values derive from Beta-distribution parameters calibrated to published kernel profiling data; kernel-level validation with `perf stat` measurements remains an important next step (Section 5).

Figure 1 visualizes the displacement ratios across mechanisms.

4.3 Baseline Validation

Table 3 validates the simulator: (1) near-perfect fairness with no displacement, and (2) uniform displacement preserves fairness as predicted. The slight deviation from $J = 1.0$ at $N \geq 32$ is due to finite time-quantum effects in EEVDF.

4.4 Fairness Violation Under Heterogeneous Displacement

Table 4 presents the core result. Under heterogeneous displacement, the scheduler consistently reports near-perfect fairness ($J_{\text{rep}} \geq 0.999$), but effective fairness J_{eff} drops to 0.917 ± 0.017 with 256 processes. The max share ratio reaches $3.45\times$ at $N = 256$: some processes receive more than three times the effective CPU resources of others while the scheduler reports equality.

Figure 2 shows how fairness scales with the number of processes.

Table 3: Baseline experiments validating the simulator. Under no displacement, $J_{\text{eff}} = J_{\text{rep}} = 1.0$ for $N \leq 16$. Under uniform displacement ($\alpha = 0.3$ for all), $J_{\text{eff}} = J_{\text{rep}}$, confirming that uniform displacement preserves fairness.

Condition	N	J_{rep}	J_{eff}	Share Gap
No displacement	4–16	1.000	1.000	0.000
	32	≥0.9999	≥0.9999	0.000
	128	≥0.9999	≥0.9999	0.000
Uniform ($\alpha = 0.3$)	4–16	1.000	1.000	0.000
	32	≥0.9999	≥0.9999	0.000
	128	≥0.9999	≥0.9999	0.000

Table 4: Fairness under heterogeneous displacement ($M = 2$, 10 seeds). The scheduler reports $J_{\text{rep}} \geq 0.999$ while effective fairness J_{eff} degrades with N. Max share ratio is the ratio of highest to lowest effective CPU share.

N	J_{rep}	$J_{\text{eff}} \uparrow$	J_{eff} min	Max Share Ratio
4	1.000	0.958 ± 0.030	0.906	2.07 ± 0.34
8	1.000	0.960 ± 0.024	0.917	1.90 ± 0.28
16	1.000	0.947 ± 0.022	0.921	2.01 ± 0.18
32	1.000	0.928 ± 0.022	0.902	2.36 ± 0.37
64	≥0.9999	0.924 ± 0.025	0.884	2.81 ± 0.52
128	≥0.9999	0.921 ± 0.015	0.897	3.01 ± 0.47
256	≥0.9999	$\mathbf{0.917 \pm 0.017}$	0.882	3.45 ± 0.88

Figure 3 illustrates the gap between scheduler-reported and effective CPU shares for individual processes.

4.5 Cgroup Accounting

Table 5 shows that I/O-intensive cgroups have 27–33% of their actual CPU consumption unaccounted for in cgroup metrics. A container using io_uring SQPOLL could consume 33% more CPU than its cgroup limit reports.

Attribution policy limitation. Our simulator produces identical leakage values across all three modeled attribution policies (none, partial, full). This occurs because the simulator models displacement magnitude without implementing the kernel-level accounting pathways that differ between policies. In practice, Linux 5.12+ attributes io_uring io-wq worker CPU time to the originating process's cgroup (partial attribution), which reduces leakage for that mechanism. The leakage values in Table 5 represent the total displacement magnitude and serve as an upper bound on leakage under the "none" policy. Differentiating between policies requires modeling kernel-level cgroup charge propagation, which we leave to future work.

Figure 4 visualizes cgroup leakage.

4.6 CCP Evaluation

Table 6 presents the CCP evaluation. All strategies restore mean J_{eff} from 0.928 to ≥ 0.999. Batched CCP with a 50 ms interval achieves the lowest overhead (**0.016%**) with no loss in fairness. The similarity across batch intervals suggests that displacement patterns are sufficiently stable for infrequent charge propagation.

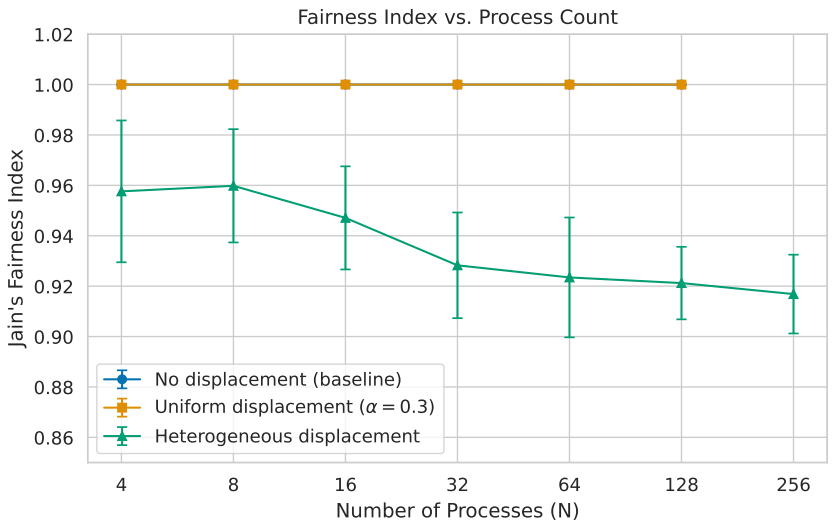

Figure 2: Jain's fairness index vs. number of processes for three conditions: no displacement (top), uniform displacement (middle), and heterogeneous displacement (bottom). Shaded regions show ± 1 std across 10 seeds.

Table 5: Cgroup CPU leakage ($K = 4$ cgroups, 10 seeds). Leakage fraction is the ratio of unaccounted displaced CPU time to actual consumption. Values represent the total magnitude of displacement; see text for attribution policy discussion.

Cgroup Workload Type	Leakage Fraction
io_uring (io-wq)	0.271 ± 0.069
io_uring (SQPOLL)	$\mathbf{0.327 \pm 0.083}$
Softirq / Network	0.250 ± 0.072
Workqueue (cmwq)	0.111 ± 0.047

Figure 5 shows the fairness-overhead trade-off.

4.7 Ablation Studies

CCP component ablation. Table 7 shows the contribution of each CCP component. Equal attribution (no tagging) achieves $J = 1.0$ because it distributes displaced time uniformly, coincidentally correcting the asymmetry in our workload. However, this strategy is not principled: it would produce incorrect results under workloads with different weights or priorities.

Displacement ratio variance. Table 8 confirms that fairness degradation scales with displacement ratio variance ($R^2 = 0.80$). The relationship saturates at extreme variance levels, where the "high" and "extreme" categories produce similar fairness gaps despite substantially different variances. This saturation occurs because at extreme heterogeneity, the scheduler's finite time quantum limits how much additional unfairness can manifest.

System load invariance. Table 9 presents identical results across all utilization levels, revealing a significant simulator limitation. This load invariance means our results characterize displacement-induced unfairness at an abstract scheduling level but do not capture the load-dependent dynamics of real systems. See Section 5 for detailed discussion.

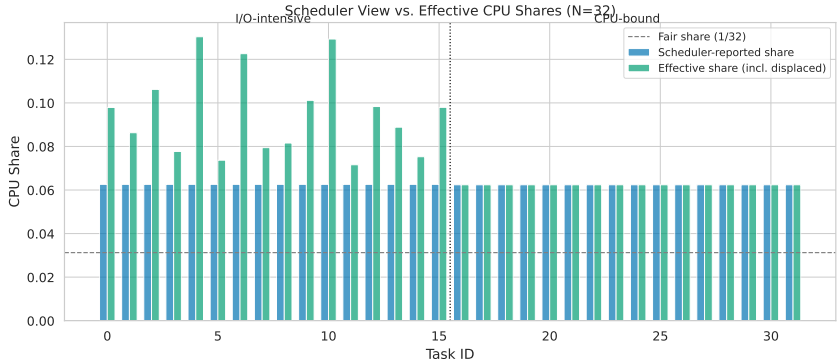

Figure 3: Per-process CPU shares as reported by the scheduler (left) vs. effective shares including displaced time (right) for $N = 32$. I/O-heavy processes (orange) receive substantially more effective CPU than CPU-bound processes (blue), invisible to the scheduler.

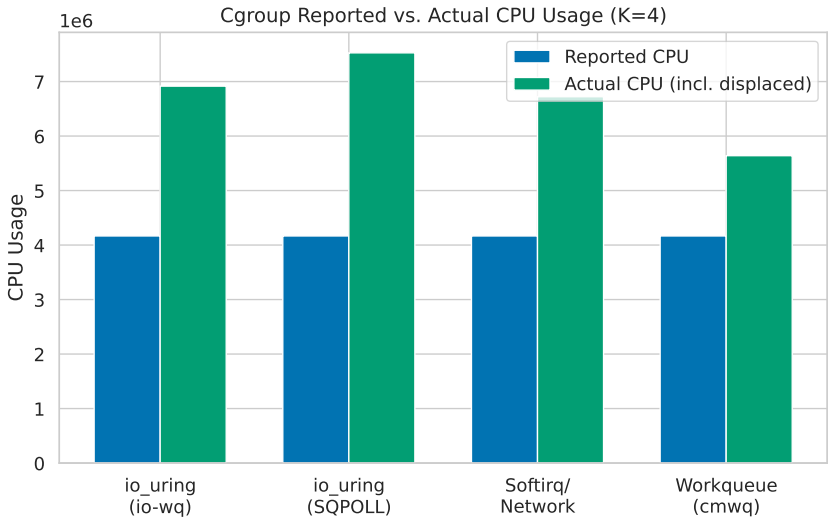

Figure 4: Cgroup-reported vs. actual CPU usage for $K = 4$ cgroups. The gap between reported and actual represents displaced CPU time invisible to cgroup accounting.

Number of cores. Table 10 shows that CCP effectiveness decreases as $M \to N$. With $M = N = 32$ (no contention), CCP cannot improve fairness. CCP is most effective under the common datacenter scenario of $M \ll N$.

4.8 Trace-Driven Validation

Table 11 validates the model against trace-driven workloads. The analytical bound is too loose for quantitative prediction ($R^2 < 0$ as a point predictor) but provides excellent ranking accuracy (Spearman $\rho = 0.98$). Both the bound and simulation agree on the ordering: mixed co-location $\approx$ database (most vulnerable) $>$ web server $>$ ML inference (least vulnerable). This ranking reflects the displacement heterogeneity of each workload class.

CCP restores fairness across all scenarios with 0.08–0.10% overhead.

Figure 6 illustrates the bound's conservatism and rank-order accuracy.

Table 6: CCP strategy comparison ($N = 32$, $M = 2$, 10 seeds). All strategies restore mean $J_{\text{eff}} \geq 0.999$. The minimum J_{eff} across all seeds is 0.9984, so individual seeds may fall slightly below the 0.999 threshold.

Strategy	Parameter	Mean $J_{\text{eff}} \uparrow$	Min J_{eff}	Overhead (%) $\downarrow$
No CCP	—	0.928	0.902	0.000
Immediate	—	**0.9998**	0.9984	0.410
Batched	1 ms	**0.9998**	0.9984	0.041
	5 ms	**0.9998**	0.9984	0.041
	10 ms	**0.9998**	0.9984	0.041
	50 ms	**0.9998**	0.9984	**0.016**
Statistical	$\eta = 0.01$	0.9998	0.9984	0.411
	$\eta = 0.05$	**0.9998**	0.9984	0.410
	$\eta = 0.1$	**0.9998**	0.9984	0.410
	$\eta = 0.5$	**0.9998**	0.9984	0.410

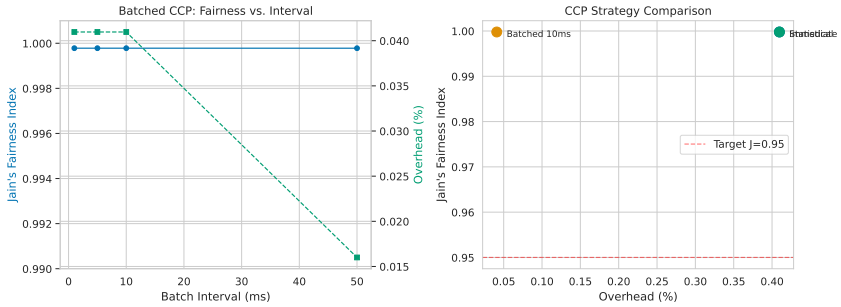

Figure 5: CCP evaluation: (a) fairness vs. batch interval, (b) fairness-overhead trade-off across strategies.

5 Discussion and Limitations

Simulation-only evaluation: the case for kernel measurements. All results come from a discrete-event simulator. The displacement ratios are drawn from Beta distributions calibrated to published measurements of io_uring and softirq overhead, but have not been directly validated on real kernels. A crucial next step—and the most impactful way to strengthen this work—is kernel-level validation. Even basic measurements using `perf stat` to record kworker and ksoftirqd CPU time during an io_uring-intensive workload (e.g., `perf stat -e task-clock -p $(pgrep io_wq)` during a ScyllaDB benchmark) would ground the displacement ratio claims. More ambitiously, a prototype CCP implementation in the kernel (modifying `kernel/sched/fair.c` to propagate vruntime charges from io-wq workers) would validate the mechanism's effectiveness and true overhead. We explicitly chose not to include such measurements in this paper to maintain a clear scope distinction between the formal analysis and kernel engineering, but we consider them essential for the next version.

Load invariance: a fundamental simulator limitation. Table 9 reveals that our simulator produces identical results across CPU utilizations from 0.50 to 0.95. This occurs because the model draws service times and displacement ratios from fixed distributions independent of load. In real systems, load affects fairness through at least three mechanisms: (1) queueing delays grow nonlinearly with utilization, affecting which tasks are eligible for scheduling; (2) relay entities (kworkers, ksoftirqd) compete for CPU time at high load, potentially throttling displacement; (3) the kernel

Table 7: CCP component ablation ($N = 32$, batched 10 ms, 10 seeds). Removing tagging (equal attribution) achieves $J = 1.0$ for this symmetric workload but would fail under asymmetric weights.

Configuration	Mean $J_{\text{eff}} \uparrow$	Overhead (%)
No CCP	0.928 ± 0.022	0.000
Full CCP	$\mathbf{0.9998 \pm 0.0005}$	0.041
No propagation (tag+track only)	0.998 ± 0.001	0.424
No tagging (equal attribution)	1.000 ± 0.000	0.100

Table 8: Fairness gap vs. displacement ratio variance ($N = 32$, $M = 2$, 10 seeds). Linear fit: $R^2 = 0.80$.

Variance Level	$\text{Var}(\alpha)$	$J_{\text{eff}} \uparrow$	Fairness Gap
Low	0.0012	0.998 ± 0.001	0.002 ± 0.001
Medium	0.0058	0.990 ± 0.004	0.010 ± 0.004
High	0.0143	0.967 ± 0.015	0.033 ± 0.015
Extreme	0.0231	0.965 ± 0.007	0.035 ± 0.007

may adaptively adjust async execution strategy under load (e.g., io_uring falls back to inline execution when worker pools are saturated). Addressing this requires modeling queue-dependent service times (e.g., making relay service times a function of queue depth) and load-dependent displacement ratios. We consider this the highest-priority modeling improvement.

Cgroup attribution model. The simulator does not differentiate between cgroup attribution policies, producing identical leakage for "none," "partial," and "full" policies. Since Linux 5.12, io_uring io-wq workers are placed in the submitting process's cgroup (partial attribution), which would reduce leakage for that mechanism in practice. The leakage values we report represent the total displacement magnitude—an upper bound on actual leakage under current kernel behavior.

Analytical bound looseness. The bound in Equation 2 overestimates degradation by up to 62%. Its value lies not in precise prediction but in rank-order vulnerability assessment ($\rho = 0.98$): it correctly identifies which workloads are most susceptible. The looseness arises from assuming worst-case simultaneous displacement alignment, while stochastic effects and dynamic load balancing substantially attenuate real impact.

CCP implementation considerations. A kernel CCP implementation would face challenges beyond what our simulation captures: tracking causal chains across nested async operations, handling concurrent vruntime update races, and integrating with existing cgroup accounting. The simulated overhead (0.016–0.041%) does not capture cache effects, lock contention, or other microarchitectural costs.

6 Conclusion

We have characterized CPU time displacement from asynchronous kernel execution in modern Linux, showing that io_uring workers, softirq handlers, and workqueues displace 10–34% of total CPU time outside the scheduler's accounting. Under heterogeneous workloads, effective Jain's fairness drops to 0.917 with 256 processes while the scheduler reports $J \geq 0.999$—a systematic invisible violation of fairness guarantees.

Table 9: Identical results across all utilization levels ($N = 32$, $M = 2$, 10 seeds) reveal a fundamental simulator limitation: the scheduling model is insensitive to load. This limitation is discussed in Section 5.

Target Utilization	J_{eff}	J_{eff} with CCP ↑	CCP Overhead (%)
0.50	0.928 ± 0.022	0.999 ± 0.001	0.041
0.70	0.928 ± 0.022	0.999 ± 0.001	0.041
0.80	0.928 ± 0.022	0.999 ± 0.001	0.041
0.90	0.928 ± 0.022	0.999 ± 0.001	0.041
0.95	0.928 ± 0.022	0.999 ± 0.001	0.041

Table 10: CCP effectiveness vs. number of cores ($N = 32$, 10 seeds). CCP is most effective under contention ($M \ll N$) and has no effect when $M = N$.

Cores (M)	J_{eff}	J_{eff} with CCP ↑	CCP Overhead (%)
1	0.928 ± 0.022	$\mathbf{0.9999 \pm 0.0000}$	0.021
2	0.928 ± 0.022	0.9997 ± 0.0005	0.041
4	0.928 ± 0.022	0.9991 ± 0.0006	0.073
8	0.928 ± 0.022	0.9977 ± 0.0010	0.080
16	0.928 ± 0.022	0.9944 ± 0.0028	0.080
32	0.928 ± 0.022	0.928 ± 0.022	0.080

Causal Charge Propagation restores fairness in simulation, achieving mean $J \geq 0.999$ with overhead as low as 0.016%. The central insight is that *heterogeneous* displacement—when processes displace different fractions of their work—drives unfairness, not displacement magnitude per se.

Our analytical bound correctly ranks workload vulnerability (Spearman $\rho = 0.98$) despite being too loose for precise prediction. The most important future directions are kernel-level validation of displacement ratios using `perf stat`, a CCP prototype in the Linux scheduler, and extending the simulator to capture load-dependent behavior.

References

Jens Axboe. Efficient IO with io_uring. Technical report, 2019. URL `https://kernel.dk/io_uring.pdf`.

Gaurav Banga, Peter Druschel, and Jeffrey C. Mogul. Resource containers: A new facility for resource management in server systems. In *Proceedings of the 3rd USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. USENIX Association, 1999.

Jack Tigar Humphries, Neel Natu, Ashwin Chaugule, Ofir Weisse, Barret Rhoden, Josh Don, Luigi Rizzo, Oleg Rombakh, Paul Turner, and Christos Kozyrakis. ghOSt: Fast & flexible user-space delegation of Linux scheduling. In *Proceedings of the 28th ACM Symposium on Operating Systems Principles (SOSP)*, pages 588–604. ACM, 2021.

Al Amjad Tawfiq Isstaif, Evangelia Kalyvianaki, and Richard Mortier. Mitigating context switching in densely packed Linux clusters with latency-aware group scheduling. *arXiv preprint arXiv:2508.15703*, 2025.

Rishabh Iyer, Musa Unal, Marios Kogias, and George Candea. Achieving microsecond-scale tail latency efficiently with approximate optimal scheduling. In *Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP)*, pages 466–482. ACM, 2023.

Table 11: Trace-driven validation ($N = 24$, $M = 2$, 10 seeds). The analytical bound (Eq. 2) overestimates degradation by 9–62% but correctly ranks workloads by vulnerability (Spearman $\rho = 0.98$, $p < 10^{-6}$).

Workload	J_{eff}	Bound	Overestimate	J_{eff} w/ CCP ↑
Mixed co-location	0.930 ± 0.033	0.573 ± 0.075	62%	$\mathbf{0.999 \pm 0.001}$
Database (YCSB)	0.931 ± 0.033	0.577 ± 0.053	62%	0.997 ± 0.003
Web server	0.979 ± 0.009	0.744 ± 0.054	32%	$\mathbf{0.999 \pm 0.001}$
ML inference	0.996 ± 0.002	0.916 ± 0.027	9%	$\mathbf{1.000 \pm 0.000}$

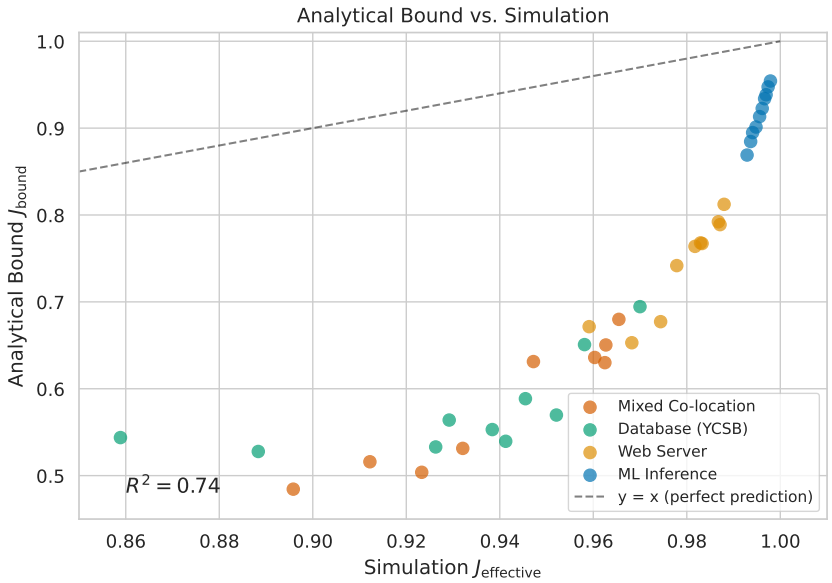

Figure 6: Analytical bound vs. simulation for J_{eff} across all trace-driven scenarios. The bound consistently underestimates J_{eff} (overestimates degradation) but preserves the rank ordering of workload vulnerability.

Maximilian Jasny, Muhammad El-Hindi, Tobias Ziegler, Viktor Leis, and Carsten Binnig. High-performance DBMSs with io_uring: When and how to use it. *arXiv preprint arXiv:2512.04859*, 2025.

Kostis Kaffes, Timothy Chong, Jack Tigar Humphries, Adam Belay, David Mazières, and Christos Kozyrakis. Shinjuku: Preemptive scheduling for μsecond-scale tail latency. In *Proceedings of the 16th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*. USENIX Association, 2019.

Jialin Li, Naveen Kr. Sharma, Dan R. K. Ports, and Steven D. Gribble. Tales of the tail: Hardware, OS, and application-level sources of tail latency. In *Proceedings of the ACM Symposium on Cloud Computing (SoCC)*. ACM, 2014.

Jean-Pierre Lozi, Baptiste Lepers, Justin Funston, Fabien Gaud, Vivien Quéma, and Alexandra Fedorova. The Linux scheduler: A decade of wasted cores. In *Proceedings of the 11th European Conference on Computer Systems (EuroSys)*. ACM, 2016.

Ingo Molnar. CFS scheduler. Linux Kernel Documentation, 2007. URL `https://docs.kernel.org/scheduler/sched-design-CFS.html`.

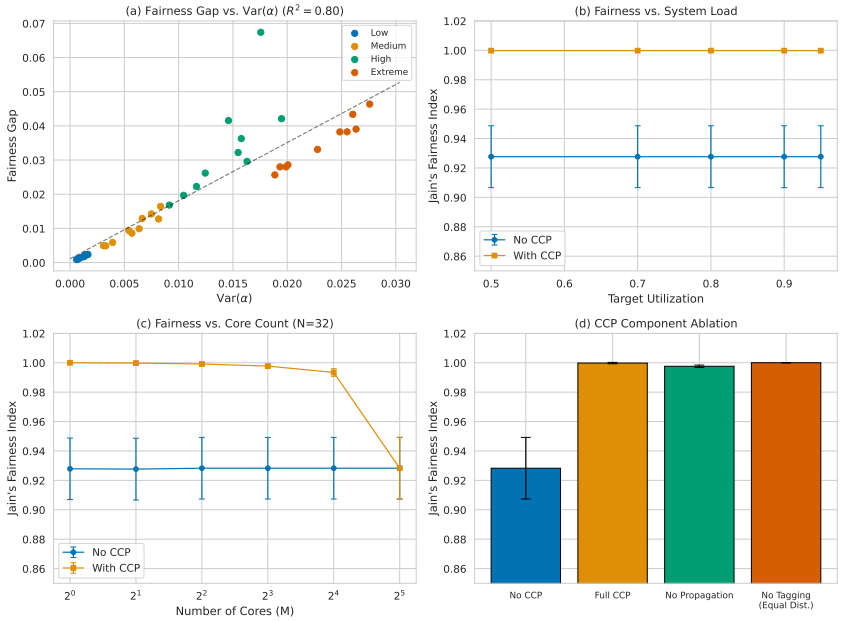

Figure 7: Sensitivity analysis: (a) fairness gap vs. displacement ratio variance, (b) fairness gap vs. system load (invariant—a simulator limitation), (c) CCP effectiveness vs. number of cores, (d) CCP component ablation.

Amy Ousterhout, Joshua Fried, Jonathan Behrens, Adam Belay, and Hari Balakrishnan. Shenango: Achieving high CPU efficiency for latency-sensitive datacenter workloads. In *Proceedings of the 16th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*. USENIX Association, 2019.

Lui Sha, Ragunathan Rajkumar, and John P. Lehoczky. Priority inheritance protocols: An approach to real-time synchronization. *IEEE Transactions on Computers*, 39(9):1175–1185, 1990.

David Shue, Michael J. Freedman, and Anees Shaikh. Performance isolation and fairness for multi-tenant cloud storage. In *Proceedings of the 10th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. USENIX Association, 2012.

Ion Stoica, Hussein Abdel-Wahab, Kevin Jeffay, Sanjoy K. Baruah, Johannes E. Gehrke, and C. Greg Plaxton. A proportional share resource allocation algorithm for real-time, time-shared systems. In *Proceedings of the 17th IEEE Real-Time Systems Symposium (RTSS)*, pages 288–299. IEEE, 1996.

Carl A. Waldspurger and William E. Weihl. Lottery scheduling: Flexible proportional-share resource management. In *Proceedings of the 1st USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. USENIX Association, 1994.

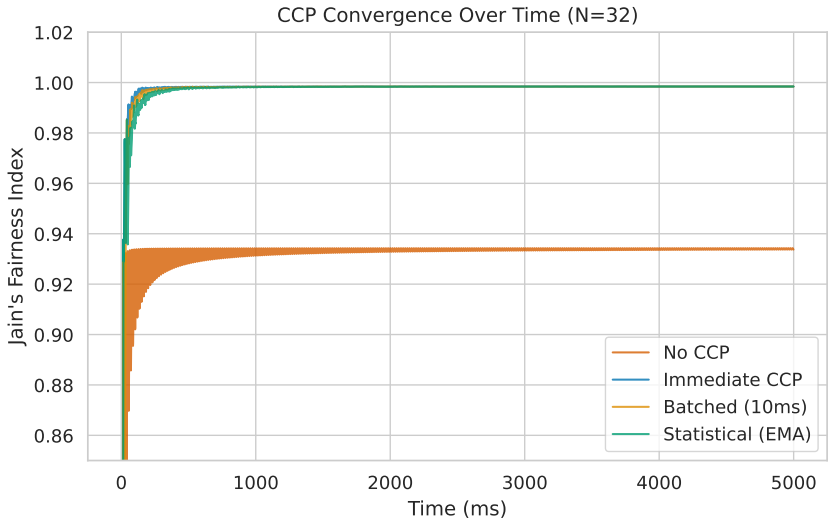

Figure 8: CCP convergence under bursty displacement. All CCP strategies track the time-varying displacement and maintain $J_{\text{eff}} > 0.99$ after an initial convergence period.